Python SPH

Distribución y empaquetado de partículas SPH
para simulaciones de impacto de ave (bird-strike)

Guía de uso y explicación de todos los códigos



Redes regulares (cúbica, FCC, hexagonal/HCP) · Colagrossi 2012 · Negi & Ramachandran 2021 · Siemann & Ritt 2019 (EWVT)
2D y 3D · geometrías analíticas y .stl · exportación a PAM-CRASH

Índice

1. Qué es este proyecto **4**

1.1 Puesta en marcha . . . 4

1.2 Orden recomendado para revisarlo . . . 4

2. Conceptos comunes a todos los scripts **5**

2.1 El bloque CONFIGURACIÓN . . . 5

2.2 Cómo cambiar la geometría . . . 5

2.3 dx, dx_eq, N, masa y h . . . 5

2.4 Qué se guarda en resultados\ . . . 6

3. Scripts 2D (00–07) **6**

00_ver_geometrias_2D.py . . . 6

01_cubica_simple_2D.py . . . 7

02_fcc_2D.py . . . 8

03_hexagonal_2D.py . . . 8

04_packing_colagrossi_2D.py . . . 8

05_packing_negi_2D.py . . . 9

06_wvt_siemann_2D.py . . . 9

07_comparar_metodos_2D.py . . . 10

4. Scripts 3D (10–19) **10**

10_ver_geometrias_3D.py . . . 10

11_cubica_simple_3D.py / 12_fcc_3D.py / 13_hexagonal_hcp_3D.py . . . 11

14_packing_colagrossi_3D.py . . . 11

15_packing_negi_3D.py . . . 12

16_wvt_siemann_3D.py . . . 12

17_comparar_metodos_3D.py . . . 12

18_visor_3D.py . . . 13

19_comprobar_stl.py . . . 13

5. Métricas de calidad: qué significan y cómo leerlas **14**

5.1 Λ: isotropía del vecindario (Siemann, paper 4) . . . 14

5.2 Γ y |∇Γ|·h (Colagrossi, paper 5) . . . 14

5.3 Error de densidad SPH (Negi, paper 6) . . . 14

5.4 Espaciado, Voronoi y superficie . . . 14

5.5 Resultados obtenidos con el pájaro . . . 15

6. Los algoritmos de empaquetado en detalle **15**

6.1 Colagrossi et al. (2012), paper 5 . . . 15

6.2 Negi & Ramachandran (2021), paper 6: algoritmo híbrido . . . 15

6.3 Extended WVT: Siemann & Ritt (2019, DLR), paper 4 . . . 16

7. Visualizadores 3D **16**

7.1 Figura de resumen (se guarda como _resumen.png) 16

7.2 Ventana interactiva (VISOR_INTERACTIVO = True) 17

7.3 Visor HTML (fichero _visor3D.html en resultados) 18

8. Usar un fichero .stl **18**

9. Exportación a PAM-CRASH **19**

10. La librería sph_common (por dentro) **20**

11. Problemas frecuentes **20**

12. Referencias **21**

1. Qué es este proyecto

El objetivo es generar, a partir de una geometría (una forma básica o un fichero **.stl**), la distribución inicial de partículas de un modelo SPH de pájaro que sea lo más estable posible. Que sea estable quiere decir que no haya reacomodo espurio de partículas al empezar el cálculo y que el primer pico de presión del impacto (Hugoniot) sea limpio. Todo está escrito solo con **numpy, scipy y matplotlib**, sin PySPH.

Se implementan y comparan seis formas de colocar las partículas:

Paso	Método	Idea	Origen
1	Red cúbica simple	Partículas a distancia dx formando cubos (cuadrados en 2D)	Clásico
2	FCC	Cubos con partícula en el centro de cada cara (como el panel SPH de HyperMesh)	HyperMesh
3	Hexagonal / HCP	Empaquetado compacto: 6 vecinos en 2D, 12 en 3D	Clásico
4	Colagrossi	Las partículas se mueven por el gradiente de Γ hasta el equilibrio, con paredes de partículas fijas	Paper 5
5	Negi (híbrido)	Gradiente de densidad + repulsión; partículas de contorno que deslizan sobre la superficie	Paper 6 (repo sph_geom)
6	EWVT	Teselación de Voronoi ponderada: repulsión decreciente desde una red agitada	Paper 4 (DLR)

Cada método tiene un script independiente que genera las partículas, calcula métricas de calidad, las dibuja y exporta el resultado para **PAM-CRASH**. Un script comparador pone todos los métodos lado a lado con las mismas métricas para poder elegir el mejor.

1.1 Puesta en marcha

- Instala las dependencias (una sola vez) desde un terminal abierto en la carpeta del proyecto:

```
pip install -r requirements.txt        # numpy, scipy, matplotlib
```

- En VS Code: *Archivo* → *Abrir carpeta* → **Escritorio\Python SPH**. Es importante abrir la **carpeta**, no un fichero suelto, porque los scripts importan la librería `sph_common` y escriben en `resultados\` de forma relativa.
- Abre un script y pulsa **F5** (o el triángulo de *Run Python File*).
- Si lo lanzas desde terminal, hazlo **desde la carpeta del proyecto**: `python 11_cubica_simple_3D.py`.
- Las ventanas de matplotlib se cierran para que el script termine. Los ficheros ya se han guardado antes de mostrarlas.

1.2 Orden recomendado para revisarlo

Día / bloque	Scripts	Objetivo
1. Geometrías	00, 10	Ver qué formas hay y cómo se definen
2. Redes 2D	01, 02, 03	Entender las redes regulares y la superficie en escalera
3. Empaquetado 2D	04, 05, 06	Ver cómo trabajan los tres algoritmos (rápido en 2D)
4. Comparación 2D	07	Leer la tabla de métricas y aprender a interpretarla
5. Redes 3D	11, 12, 13	Las mismas redes en el pájaro 3D y los visores 3D
6. Empaquetado 3D	14, 15, 16	Los algoritmos sobre el pájaro de Siemann (5.450 partículas)
7. Comparación 3D	17, 18	Elegir método y revisar resultados guardados
8. Tu geometría	19 + cualquiera	Comprobar y usar un .stl propio

Atajo
Todos los resultados de la cápsula/pájaro (2D y 3D) ya están calculados en `resultados\`. Puedes abrir las figuras y los visores HTML antes de ejecutar nada, y el comparador los reutiliza sin recalcular.

2. Conceptos comunes a todos los scripts

2.1 El bloque CONFIGURACIÓN

Cada script empieza con un bloque como este. **Es lo único que hay que tocar** para cambiar la geometría, la resolución o los parámetros del método:

```
# ------------------------------ CONFIGURACION ----------------------------
GEOMETRIA = "capsula"     # esfera, cubo, caja, cilindro, capsula, elipsoide, stl
PARAMS_GEOM = {}          # dimensiones; {} = valores por defecto
DX = 6.65                 # distancia entre particulas [mm]
RHO0 = 1.1317e-6          # densidad [kg/mm^3]
H_FACTOR = 1.2            # h = H_FACTOR * dx (smoothing length para la simulacion)
MOSTRAR = True            # abrir las figuras al terminar
VISOR_INTERACTIVO = True  # (3D) ventana con deslizador de corte
ABRIR_HTML = False        # (3D) abrir el visor HTML en el navegador
```

2.2 Cómo cambiar la geometría

GEOMETRIA	PARAMS_GEOM (ejemplo)	Notas
`"esfera"`	`{"R": 50}`	
`"cubo"`	`{"L": 100}`	
`"caja"`	`{"Lx": 150, "Ly": 100, "Lz": 60}`	
`"cilindro"`	`{"R": 50, "L": 150}`	Eje X
`"capsula"`	`{"R": 53.5}`	Pájaro de Siemann: longitud total 4R, eje X
`"capsula"`	`{"R": 50, "L_cilindro": 150}`	Pájaro más alargado
`"elipsoide"`	`{"a": 80, "b": 50, "c": 50}`	Semiejes en X, Y, Z
`"stl"`	`{"archivo": "stl/mi.stl", "escala": 1.0, "centrar": True}`	Ver capítulo 8

En 2D (scripts 00–07): `circulo(R)`, `cuadrado(L)`, `rectangulo(L, H)`, `triangulo(L)`, `elipse(a, b)`, `hexagono(R)`, `estrella(R, r, puntas)`, `capsula(R)` y `poligono(vertices=[(x, y), ...])`. Todas las geometrías están centradas en el origen y en mm.

Importante
Usa la **misma GEOMETRIA, PARAMS_GEOM y DX** en todos los scripts que quieras comparar: el comparador busca los resultados por el nombre de la geometría y del método.

2.3 dx, dx_eq, N, masa y h

- **DX**: distancia nominal entre partículas que eliges tú.
- **N**: número de partículas que resulta. Depende del método: las redes recortadas no llenan exactamente el volumen, mientras que la WVT impone N = V/dx³ exacto.
- **dx_eq = (V/N)$^{1/d}$**: espaciado equivalente real (d = 2 o 3). Es el que se usa en las métricas.
- **Masa**: **todas las partículas tienen la misma masa**, m = ρ0·V/N, de modo que la masa total es exactamente la del cuerpo (1,814 kg para el pájaro de Siemann con ρ0 = 1,1317·10^{-6} kg/mm³). En 2D, V = área × espesor, con espesor = dx_eq.
- **h** (smoothing length para la simulación) = H_FACTOR·dx_eq, 1,2 por defecto. Siemann usa h = 1,9 × radio de partícula ≈ 1,18·dx. Los métodos de empaquetado usan internamente su propia h,

independiente de esta.

- **MISMO_VOLUMEN** (redes): con True, el paso de la red se ajusta para que cada partícula ocupe dx^3. Así todas las redes tienen la misma N y la misma masa, y la comparación es justa. Con False, el lado de la celda es dx, que es el *pitch* de HyperMesh.

2.4 Qué se guarda en resultados\

Fichero	Contenido
`<geom>_<metodo>_resumen.png`	Figura de resumen: partículas, mapas de Λ, error de densidad, distancias, histogramas y métricas
`<geom>_<metodo>_convergencia.png`	Historial de convergencia (solo métodos iterativos)
`<geom>_<metodo>_metricas.json`	Todas las métricas numéricas
`<geom>_<metodo>.npz`	Posiciones (las reutilizan el comparador y el visor 18)
`<geom>_<metodo>_pamcrash.inc`	Tarjetas NODE + SPHEL para PAM-CRASH (capítulo 9)
`<geom>_<metodo>.csv`	x, y, z, masa, h
`<geom>_<metodo>_visor3D.html`	(3D) Visor interactivo para el navegador
`<geom>_<metodo>.vtk`	(3D) Para ParaView, con las métricas como campos
`comparacion_<geom>.* / comparacion3D_<geom>.*`	Salida de los comparadores 07 y 17

3. Scripts 2D (00–07)

La fase 2D sirve para entender los métodos con un coste de cálculo muy bajo (segundos o un minuto). Por defecto usan la cápsula 2D (pájaro en sección) con dx = 4 mm, unas 1.260 partículas.

00_ver_geometrias_2D.py

Dibuja las nueve geometrías 2D disponibles. Para cada una muestra el contorno (negro), el contorno desplazado dx/2 hacia dentro (rojo, donde irá la primera capa de partículas en Negi) y las esquinas convexas que se fijan como *hard points* (azul).

Parámetro	Valor por defecto	Qué controla
`DX`	`4.0`	Solo afecta al desplazamiento dx/2 dibujado
`GEOMETRIAS`	`las 9`	Diccionario nombre → parámetros; añade o quita formas aquí

Qué vas a ver al ejecutarlo

- Una cuadrícula 3×3 con todas las formas y su área.

Tiempo orientativo: 1 s

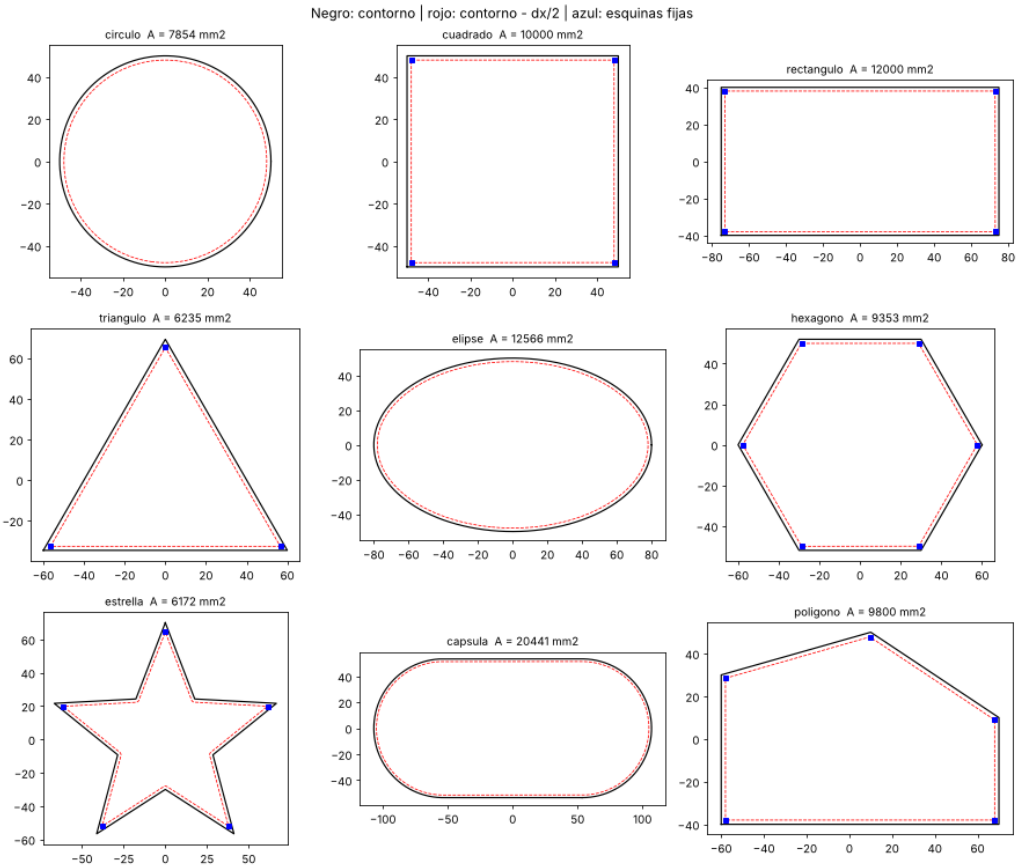


Figura 1. Geometrías 2D (script 00).

01_cubica_simple_2D.py

Paso 1. Red cuadrada de paso dx recortada a la geometría: se quedan los nodos cuyo centro cae dentro. Es la referencia clásica.

Parámetro	Valor por defecto	Qué controla
`GEOMETRIA / PARAMS_GEOM`	`capsula / {}`	Forma y dimensiones
`DX`	`4.0`	Paso de la red [mm]
`RH00`	`9.5e-7`	Densidad [kg/mm³] (2D)
`H_FACTOR`	`1.2`	h = H_FACTOR·dx_eq
`DESFASE`	`(0.5, 0.5)`	Posición de la red respecto al origen (fracción de dx); cambia cómo se recorta el borde

Qué vas a ver al ejecutarlo

- Figura de resumen: partículas a tamaño real, Λ, error de densidad, distancia entre vecinos, |∇Γ|·h, histogramas y tabla de métricas.
- En consola, la misma tabla de métricas y las rutas de los ficheros exportados.

Qué comprobar

- El contorno en escalera (columnas «hueco» y «capa exterior» de las métricas).
- Prueba DESFASE = (0, 0) y verás cómo cambia el borde.

Tiempo orientativo: 1 s

02_fcc_2D.py

Paso 2. Celda cuadrada con una partícula en cada esquina y otra en el centro, el equivalente 2D del FCC de HyperMesh. **En 2D esto es exactamente una red cuadrada girada 45°.** La diferencia real entre FCC y cúbica aparece en 3D.

Parámetro	Valor por defecto	Qué controla
`MISMO_VOLUMEN`	`True`	True: lado a = √2·dx (mismo N que la cúbica). False: a = dx (pitch de HyperMesh, el doble de partículas)
`DESFASE`	`(0.5, 0.5)`	Posición de la red

Qué vas a ver al ejecutarlo

- La red girada 45° y un borde en escalera distinto al de la cúbica.

Tiempo orientativo: 1 s

03_hexagonal_2D.py

Paso 3. Red hexagonal compacta (triangular): cada partícula tiene 6 vecinos a la misma distancia, el empaquetado más denso en 2D. Con VARIANTE = "panal" genera la red de panal de abeja (3 vecinos), que Colagrossi llama «hexagonal» y que demuestra **siempre inestable** (tabla 1 del paper 5). Se incluye solo para verlo.

Parámetro	Valor por defecto	Qué controla
`VARIANTE`	`"triangular"`	"triangular" (compacta) o "panal"
`MISMO_VOLUMEN`	`True`	Paso s = dx·√(2/√3) ≈ 1,075·dx para que cada partícula ocupe dx²

Qué vas a ver al ejecutarlo

- Filas desplazadas medio paso; vecino más próximo a 1,065·dx (con MISMO_VOLUMEN).

Tiempo orientativo: 1 s

04_packing_colagrossi_2D.py

Paso 4. Algoritmo de Colagrossi et al. (2012). Se parte de una red y se integra un sistema dinámico ficticio en el que cada partícula se mueve hacia donde «falta masa» (−∇Γ) con amortiguamiento, hasta el equilibrio. El contorno del cuerpo se convierte en una pared de partículas **fijas** colocadas en curvas paralelas al contorno a dx/2, 3dx/2, 5dx/2… para que la superficie quede lisa. Detalle en el apartado 6.1.

Parámetro	Valor por defecto	Qué controla
`RED_INICIAL`	`"cubica"`	Red de partida: "cubica" o "hexagonal"
`H_FACTOR`	`2.0`	h/dx del kernel Wendland durante el empaquetado (tabla 1 del paper: estable con 2,0)
`ALPHA`	`5e-3`	Amortiguamiento adimensional (paper: 1e-3 a 5e-3). Más alto, converge antes
`MAX_ITER`	`4000`	Iteraciones máximas
`TOL_EK`	`1e-7`	Parada: energía cinética específica < TOL_EK·β
`H_FACTOR_SIM`	`1.2`	h que se exporta para la simulación

Qué vas a ver al ejecutarlo

- Mensajes de progreso: `it, Ek/beta, max|gradGamma|h`; ambos deben bajar.
- Figura de convergencia (_convergencia.png) y figura de resumen.

Qué comprobar

- Que Ek/β baje varios órdenes de magnitud.
- En el resumen, |∇Γ|·h casi nulo en el interior: es la «fuerza residual» que quedaría en t = 0.

Tiempo orientativo: 15–60 s

05_packing_negi_2D.py

Paso 5. Algoritmo híbrido de Negi & Ramachandran (2021), el del repositorio sph_geom, reescrito sin PySPH. Partículas libres en red hexagonal llenan una caja que envuelve el cuerpo y van rodeadas de partículas congeladas. Cada 50 iteraciones, las libres que se acercan a la superficie (desplazada dx/2 hacia dentro) se convierten en partículas de contorno, que solo pueden deslizar sobre ella. Al final se separa el interior. Detalle en el apartado 6.2.

Parámetro	Valor por defecto	Qué controla
HDX	1.2	h/dx (kernel quintic spline, como el paper)
TOL	1e-4	Parada: max|u|·dt/h < TOL (ec. 23 del paper)
MAX_ITER	8000	Iteraciones máximas
FREQ_PROY	50	Cada cuántas iteraciones se proyectan partículas al contorno
VER_CAJA_COMPLETA	True	Dibuja también libres exteriores, congeladas y esquinas

Qué vas a ver al ejecutarlo

- Una figura extra con todas las partículas de la caja: libres (azul), contorno (rojo), congeladas (verde), esquinas (negro).
- Progreso: `max|u|dt/h, contorno = n/Ns, Linf(rho-1)`.

Qué comprobar

- Que el número de partículas de contorno se acerque a Ns.
- La capa exterior perfectamente pegada a dx/2 de la superficie.

Tiempo orientativo: 1–2 min

06_wvt_siemann_2D.py

Paso 6. Weighted Voronoi Tessellation extendida (EWVT), el método del DLR (Siemann & Ritt 2019) para pájaros SPH en PAM-CRASH/VPS. Parte de una red cúbica agitada aleatoriamente hasta un 50 % de dx y aplica desplazamientos repulsivos entre vecinos cuya intensidad ξ decrece en cada iteración. La superficie actúa como contacto rígido a dx/2. N = A/dx² exacto. Detalle en el apartado 6.3.

Parámetro	Valor por defecto	Qué controla
H_FACTOR_WVT	1.0	h del WVT respecto a dx (soporte 2h)
XI	4e-2	Intensidad inicial ξ (tabla 1 del paper)
KAPPA	4e-4	Reducción por iteración: ξ ← (1−κ)ξ
CHI	1e-2	Evita la singularidad cuando r → 0
CR	1e-8	Parada: ΔR/ΔR_max ≤ CR
AGITAR	0.5	Agitado inicial (fracción de dx)
SEMILLA	0	Cambiándola obtienes otra distribución aleatoria

Qué vas a ver al ejecutarlo

- Progreso: dR/dRmax y ξ.
- Figura de resumen y de convergencia.

Qué comprobar

- Distribución «aleatoria» isótropa con Λ medio ~0,98.
- Prueba varias SEMILLAS: las métricas deben ser parecidas.

Tiempo orientativo: 10–20 s

07_comparar_metodos_2D.py

Postprocesado. Para la misma geometría y dx, carga los resultados guardados por 01–06 (o los calcula si no existen), calcula las métricas de todos y los compara.

Parámetro	Valor por defecto	Qué controla
METODOS	los 6	Lista de métodos a comparar
USAR_GUARDADOS	True	Reutiliza resultados\<geom>_<metodo>.npz si existe con el mismo DX
GUARDAR_NUEVOS	True	Guarda los que tenga que calcular

Qué vas a ver al ejecutarlo

- Tabla en consola, con un asterisco (*) en el mejor valor de cada fila y la explicación de cada métrica.
- comparacion_<geom>.csv (se abre en Excel, separador «;»).
- Figuras: partículas y mapas de Λ lado a lado, barras por métrica e histogramas superpuestos.

Tiempo orientativo: Inmediato si todo está guardado

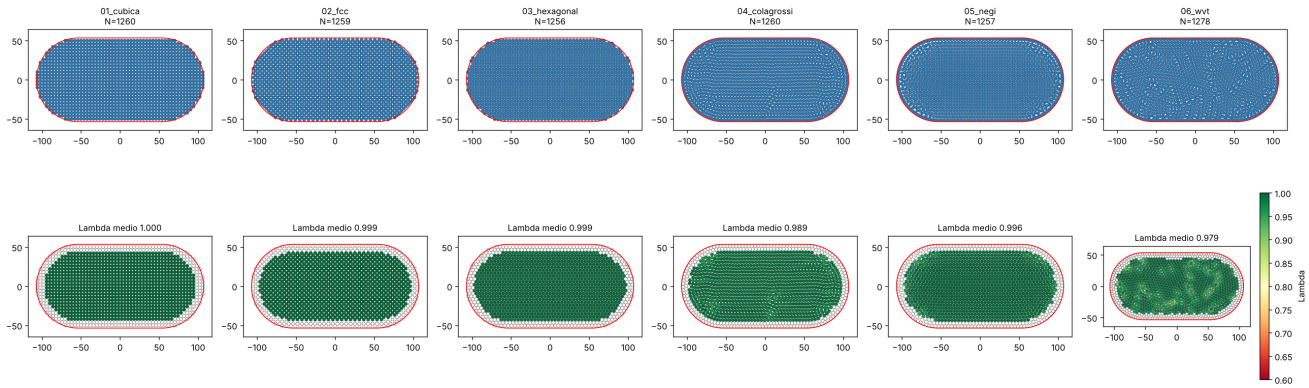


Figura 2. Comparación 2D (script 07). Arriba, las partículas; abajo, Λ en el interior. Las redes tienen el borde en escalera; los métodos de empaquetado siguen el contorno.

4. Scripts 3D (10–19)

Mismos pasos en 3D. Por defecto se usa el pájaro de Siemann: cápsula de R = 53,5 mm y longitud 214 mm ($1{,}604 \cdot 10^6$ mm³, 1,814 kg) con dx = 6,65 mm, que da unas 5.450 partículas, el modelo «grueso» del paper. El modelo fino del paper (40.824 partículas) corresponde a dx ≈ 3,4 mm y tarda unas 8 veces más.

10_ver_geometrias_3D.py

Dibuja las geometrías 3D básicas (malla gris) con los puntos de la superficie desplazada dx/2 hacia dentro (rojo), repartidos uniformemente.

Parámetro	Valor por defecto	Qué controla
DX	6.65	Separación de los puntos dibujados

Qué vas a ver al ejecutarlo

- Seis vistas 3D: esfera, cubo, caja, cilindro, cápsula y elipsoide.

Tiempo orientativo: ~10 s

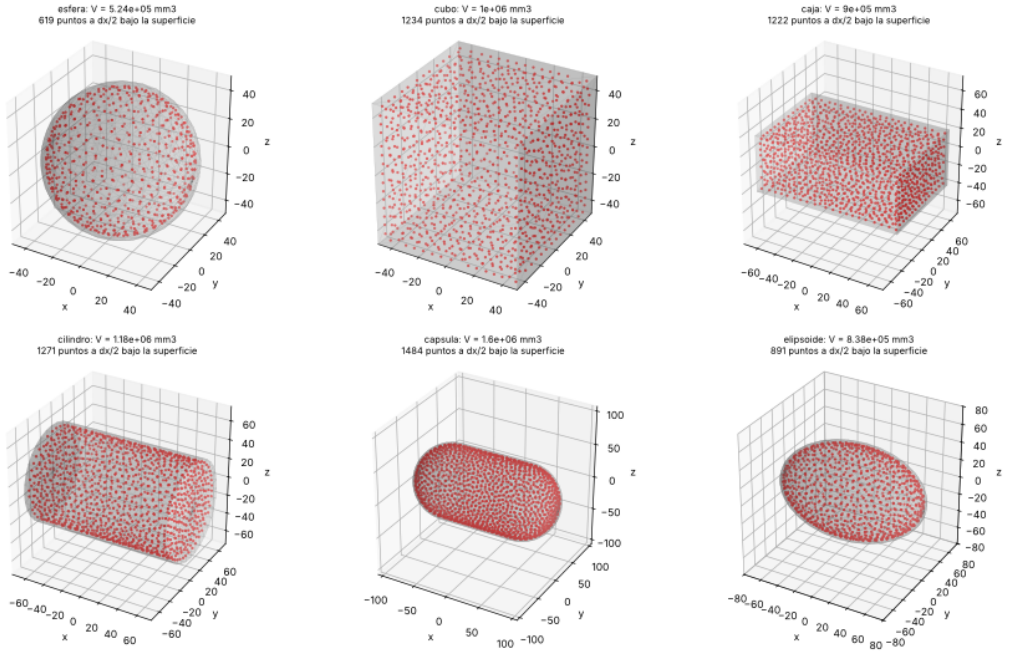


Figura 3. Geometrías 3D (script 10).

11_cubica_simple_3D.py / 12_fcc_3D.py / 13_hexagonal_hcp_3D.py

Pasos 1–3 en 3D. **11**: cúbica simple (6 vecinos). **12**: FCC (4 partículas por celda, 12 vecinos), la opción «face centered cubic» de HyperMesh. **13**: HCP, hexagonal compacta con capas ABAB apiladas a lo largo del eje X (12 vecinos, tan compacta como la FCC). Con RED = "bcc" genera la cúbica centrada en el cuerpo.

Parámetro	Valor por defecto	Qué controla
`DX`	`6.65`	Paso [mm]
`RHO0`	`1.1317e-6`	kg/mm³ (pájaro de 1,814 kg)
`MISMO_VOLUMEN`	`True`	(12, 13) Paso ajustado para que cada partícula ocupe dx³
`RED`	`"hcp"`	(13) "hcp" o "bcc"
`VISOR_INTERACTIVO`	`True`	Abre la ventana con deslizador de corte
`ABRIR_HTML`	`False`	Abre el visor HTML automáticamente

Qué vas a ver al ejecutarlo

- Figura de resumen 3D (apartado 7.1), la ventana interactiva (7.2) y, en resultados\, el visor HTML (7.3).

Qué comprobar

- En las secciones, la superficie en escalera: capa exterior irregular en un 60–67 %.

Tiempo orientativo: 5 s cada uno

14_packing_colagrossi_3D.py

Paso 4 en 3D. Las capas de partículas fijas se colocan sobre superficies paralelas a la del cuerpo, muestreadas uniformemente.

Parámetro	Valor por defecto	Qué controla
`RED_INICIAL`	`"cubica"`	"cubica", "hexagonal" (HCP) o "fcc"

Parámetro	Valor por defecto	Qué controla
H_FACTOR	1.4	En 3D se usa 1,4 en lugar de 2,0 por coste (con 2,0 habría ~270 vecinos por partícula)
ALPHA / MAX_ITER / TOL_EK	5e-3 / 3000 / 1e-7	Igual que en 2D

Qué vas a ver al ejecutarlo

- Progreso de Ek/β y |∇Γ|h; figuras y visores.

Tiempo orientativo: 4–8 min

15_packing_negi_3D.py

Paso 5 en 3D, con dos cambios respecto al paper para que sea viable en un PC. Las dos opciones se pueden volver a la versión del paper desde la configuración (unas 5 veces más lenta).

Parámetro	Valor por defecto	Qué controla
KERNEL / HDX	"wendland" / 1.3	Paper: "quintic" / 1.2 (unas 2,6 veces más vecinos)
CONTENEDOR	"forma"	"forma": las libres ocupan el cuerpo más una banda de MARGEN·dx. "caja": como el paper
MARGEN	2	Banda de partículas libres fuera del cuerpo (en dx)
TOL / MAX_ITER / MIN_ITER	1e-4 / 4000 / 1000	Convergencia

Qué vas a ver al ejecutarlo

- Progreso con el número de partículas de contorno frente a Ns.

Qué comprobar

- El centro del pájaro queda ~4 % más denso que la media (ver capítulo 5). Es una característica conocida del método: el propio paper habla de variaciones del 2–3 %.

Tiempo orientativo: 3–6 min

16_wvt_siemann_3D.py

Paso 6 en 3D, con los parámetros de la tabla 1 del paper.

Parámetro	Valor por defecto	Qué controla
H_FACTOR_WVT / XI / KAPPA / CHI	1.0 / 4e-2 / 4e-4 / 1e-2	Igual que en 2D
CR / MAX_ITER	1e-8 / 6000	Convergencia
AGITAR / SEMILLA	0.5 / 0	Inicio aleatorio

Qué vas a ver al ejecutarlo

- Progreso y visores; N = V/dx³ exacto (5.453 con dx = 6,65).

Tiempo orientativo: 1–3 min

17_comparar_metodos_3D.py

Comparador 3D: igual que el 07 pero con 11–16. Carga los .npz guardados si existen.

Parámetro	Valor por defecto	Qué controla
METODOS	los 6	11_cubica ... 16_wvt
USAR_GUARDADOS	True	Reutilizar resultados

Qué vas a ver al ejecutarlo

- Tabla en consola, comparacion3D_<geom>.csv, figura de secciones (y = 0 y x = 0 con Λ), barras e histogramas.

Tiempo orientativo: Inmediato si están guardados

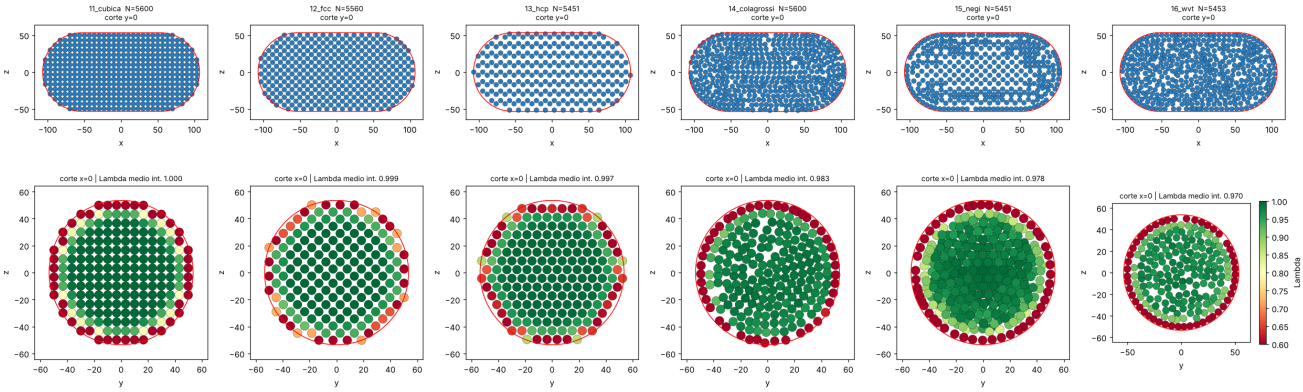


Figura 4. Comparación 3D (script 17). Arriba, la sección longitudinal y = 0; abajo, la sección transversal x = 0 coloreada con Λ. Se ve la superficie en escalera de las redes frente a la capa exterior regular de los métodos de empaquetado.

18_visor_3D.py

Abre cualquier resultado guardado en los dos visores interactivos sin recalcular nada. Es útil para revisar resultados de días anteriores.

Parámetro	Valor por defecto	Qué controla
METODO	"16_wvt"	Uno de 11_cubica, 12_fcc, 13_hcp, 14_colagrossi, 15_negi, 16_wvt
ABRIR_HTML / VENTANA_MATPLOTLIB	True / True	Qué visores abrir

Tiempo orientativo: 5 s

19_comprobar_stl.py

Comprueba un .stl antes de usarlo: si la malla está cerrada, el volumen, el área, el tamaño, la masa con RHO0 y cuántas partículas saldrán con varios DX. Dibuja la malla, la capa de contorno y tres secciones con la distancia firmada (azul = dentro). Ver capítulo 8.

Parámetro	Valor por defecto	Qué controla
ARCHIVO	"stl/ejemplo_pajaro_capsula.stl"	Tu fichero dentro de la carpeta stl
ESCALA	1.0	1000 si el STL está en metros
CENTRAR	True	Mueve el centro de masas al origen

Tiempo orientativo: 10–30 s

5. Métricas de calidad: qué significan y cómo leerlas

Todas se calculan en `sph_common/metricas.py` con el kernel Wendland C2 y la h de la simulación (1,2·dx_eq). Las métricas «interiores» usan solo partículas con el soporte del kernel completo dentro del cuerpo (a más de 2h de la superficie), como en Siemann. Las de superficie miden la primera capa, que es la que golpea.

5.1 Λ: isotropía del vecindario (Siemann, paper 4)

$$I_i^{ab} = \sum_j \frac{m_j}{\rho_j}(x_i^a - x_j^a)(x_i^b - x_j^b)\,W_{ij} \qquad \Lambda_i = \frac{\lambda_{min}(I_i)}{\lambda_{max}(I_i)}$$

Λ = 1 indica que los vecinos están repartidos igual en todas las direcciones. Siemann obtiene 0,83 ≤ Λ ≤ 1 con EWVT, mientras que en el modelo mallado de referencia más del 90 % de las partículas queda por debajo de 0,9. **Una red perfecta también da Λ = 1**: Λ no penaliza las redes; lo que las penaliza es la superficie y las direcciones preferentes.

5.2 Γ y |∇Γ|·h (Colagrossi, paper 5)

$$\Gamma_i = \sum_j V_j W_{ij} \qquad \nabla\Gamma_i = \sum_j V_j \nabla_i W_{ij}$$

Γ = 1 y ∇Γ = 0 en una distribución en equilibrio. **|∇Γ|·h es la fuerza que movería las partículas en el primer paso de la simulación**, la responsable del reacomodo espurio y del ruido inicial de presión. Cuanto más bajo, mejor.

5.3 Error de densidad SPH (Negi, paper 6)

$$\rho_i = \sum_j m_j W_{ij} \qquad \text{err} = \frac{\rho_i - \rho_0}{\rho_0}$$

Se dan tres valores en el interior: **L∞** (máximo), **desviación típica** (falta de uniformidad) y **sesgo medio** (precompresión global). Parte del sesgo es el error de cuadratura propio del kernel con h = 1,2·dx, que aparece incluso en una red perfecta.

5.4 Espaciado, Voronoi y superficie

Métrica	Definición	Ideal
dnn_min, dnn_CV_%	Distancia al vecino más cercano / dx_eq: mínima y dispersión (Siemann fig. 6)	≈1 y 0 %
vecinos_media	Vecinos dentro del soporte 2h	~17 (2D), ~50 (3D)
Vvor_max/min, Vvor_CV_%	Volumen de la celda de Voronoi / V0 en el interior (Siemann fig. 5)	1 y 0 %
hueco_sup_media / _max	Distancia de cada punto de la superficie a la partícula más cercana / dx	0,5
capa_ext_CV_%	Irregularidad de la profundidad de la primera capa (superficie en escalera)	0

5.5 Resultados obtenidos con el pájaro

Métrica (3D, dx = 6,65)	Cúbica	FCC	HCP	Colagrossi	Negi	WVT
N	5600	5560	5451	5600	5451	5453
Λ medio interior	1.000	0.999	0.997	0.984	0.978	0.970
Λ mínimo interior	0.993	0.986	0.980	0.958	0.924	0.933
|∇Γ|·h máx	~0	~0	~0	0.016	0.043	0.032
desv. densidad %	~0	~0	~0	0.19	2.05	0.55
sesgo densidad %	−0.85	−0.67	1.21	1.39	4.27	0.35
dnn mínimo / dx	1.01	1.13	1.12	0.89	0.90	0.90
hueco superficie máx / dx	0.95	1.09	1.09	0.90	0.86	0.87
capa exterior CV %	65.5	66.6	60.1	21.7	≈0	≈0

Lectura rápida
Redes: interior perfecto pero superficie en escalera, que es justo la nariz que impacta. **Colagrossi**: el interior más uniforme, con la superficie algo menos regular. **WVT**: superficie perfecta, el menor sesgo de densidad y el más rápido, y además es el método validado por el DLR en PAM-CRASH/VPS. **Negi**: superficie perfecta pero un gradiente de densidad radial de ~4 %. La decisión final la dará la simulación: el primer pico y el ruido de presión en PAM-CRASH.

6. Los algoritmos de empaquetado en detalle

6.1 Colagrossi et al. (2012), paper 5

Con presión, densidad y volumen constantes, la ecuación de cantidad de movimiento SPH se reduce a un sistema que lleva las partículas al mínimo de una energía potencial (ec. 2.16 del paper):

$$\frac{d\mathbf{u}_i}{dt} = -\beta\,\nabla\Gamma_i - \zeta\,\mathbf{u}_i, \qquad \frac{d\mathbf{r}_i}{dt} = \mathbf{u}_i, \qquad \beta = \frac{2p_0}{\rho_0} = 1$$

$$\zeta = \alpha\,\frac{\sqrt{\beta}}{V_0^{1/d}}, \qquad \Delta t = \mathrm{CFL}\,\frac{V_0^{1/d}}{\sqrt{\beta}}, \qquad V_0 = \frac{V}{N}$$

- $-\nabla\Gamma$ apunta hacia donde «falta masa», así que las partículas rellenan huecos y se alejan de las zonas densas.
- El contorno se modela con **partículas fijas** que hacen de pared. Aquí van en capas paralelas a la superficie (a dx/2, 3dx/2, ...) en lugar de en red, para que el borde quede liso.
- Al ser un sistema lagrangiano con disipación, siempre converge a un equilibrio estable (sección 2.1 del paper).
- Parada: energía cinética específica < TOL_EK·β.
- Implementación: `empaquetar_colagrossi()` en `sph_common/packing.py`.

6.2 Negi & Ramachandran (2021), paper 6: algoritmo híbrido

$$\frac{d\mathbf{u}_i}{dt} = -\frac{p_b}{m_i}\sum_j V_i V_j \nabla W_{ij} \;-\; \nabla\phi_{RF} \;-\; \zeta\,\mathbf{u}_i$$

$$\phi_{RF} = 12\,k_r\left(\frac{c^2}{r^3} - \frac{c}{r^2}\right), \quad c = \frac{2\alpha\Delta s}{3},\ \alpha = 0.95 \quad (\text{repulsion solo si } r < \alpha\Delta s)$$

- **Libres**: red hexagonal (HCP en 3D) dentro y fuera del cuerpo. **Congeladas**: rodean a las libres para darles soporte de kernel.

- **Superficie de contorno**: el contorno desplazado Δs/2 hacia dentro (ec. 11), porque en SPH la frontera real queda a medio espaciado de la última partícula.
- Cada 50 iteraciones (algoritmo 1), las libres a menos de 0,5Δs (0,65Δs tras 4 pasadas vacías) se convierten en **partículas de contorno**: se proyectan sobre la superficie y solo deslizan tangencialmente.
- Esquinas convexas (2D): se fija una partícula en la esquina (*hard point*).
- Constantes del código de referencia: $p_b = 1$, $\zeta = 0{,}2/\Delta s$ (2D) o $0{,}5/\Delta s$ (3D), $k_r = 0{,}004\Delta s$ (2D) o $0{,}006\Delta s$ (3D), $\Delta t = 0{,}1h/\sqrt{p_b}$.
- Parada: $\max|u|\cdot\Delta t/h < tol$. Al final las partículas interiores y las de contorno forman el cuerpo.
- Implementación: `empaquetar_negi()`.

6.3 Extended WVT: Siemann & Ritt (2019, DLR), paper 4

$$\Delta\mathbf{r}_i = \xi\, h_i \sum_j f(h_{ij}, r_{ij})\,\frac{\mathbf{r}_{ij}}{r_{ij}}, \qquad f = \frac{1}{(r_{ij}/h_{ij}+\chi)^2} - \frac{1}{(2+\chi)^2} \quad (r_{ij} < 2h_{ij})$$

$$\xi_{n+1} = (1-\kappa)\,\xi_n, \qquad \text{parada}: \ \frac{\Delta R_n}{\Delta R_{max}} \le C_R, \quad \Delta R = \sum_i |\Delta\mathbf{r}_i|^2$$

- Es un método geométrico, no dinámico: cada partícula se aparta de sus vecinos con una fuerza que decae como $1/r^2$ (Diehl et al. 2015).
- Inicio: red cúbica agitada (*node shaking*) hasta un 50 % de dx, que según el paper converge antes que una red perfecta.
- Contorno: contacto rígido a una altura igual al radio de partícula (dx/2). En el paper es un contacto de penalización contra una malla de shells en VPS.
- Admite h variable, $h_i = h(x)$: es la base natural para la fase de «inflation» (resolución variable).
- Implementación: `empaquetar_wvt()` (argumento `h_func` para h variable).

7. Visualizadores 3D

7.1 Figura de resumen (se guarda como _resumen.png)

Arriba: medio cuerpo en 3D ($y \le 0$) coloreado por profundidad, y tres **secciones** con las partículas a tamaño real: longitudinal $y = 0$ (distancia entre vecinos), horizontal $z = 0$ (Λ interior) y transversal $x = 0$ (error de densidad). Abajo: histogramas, el espaciado a lo largo del eje del pájaro (como la fig. 6 de Siemann) y la tabla de métricas.

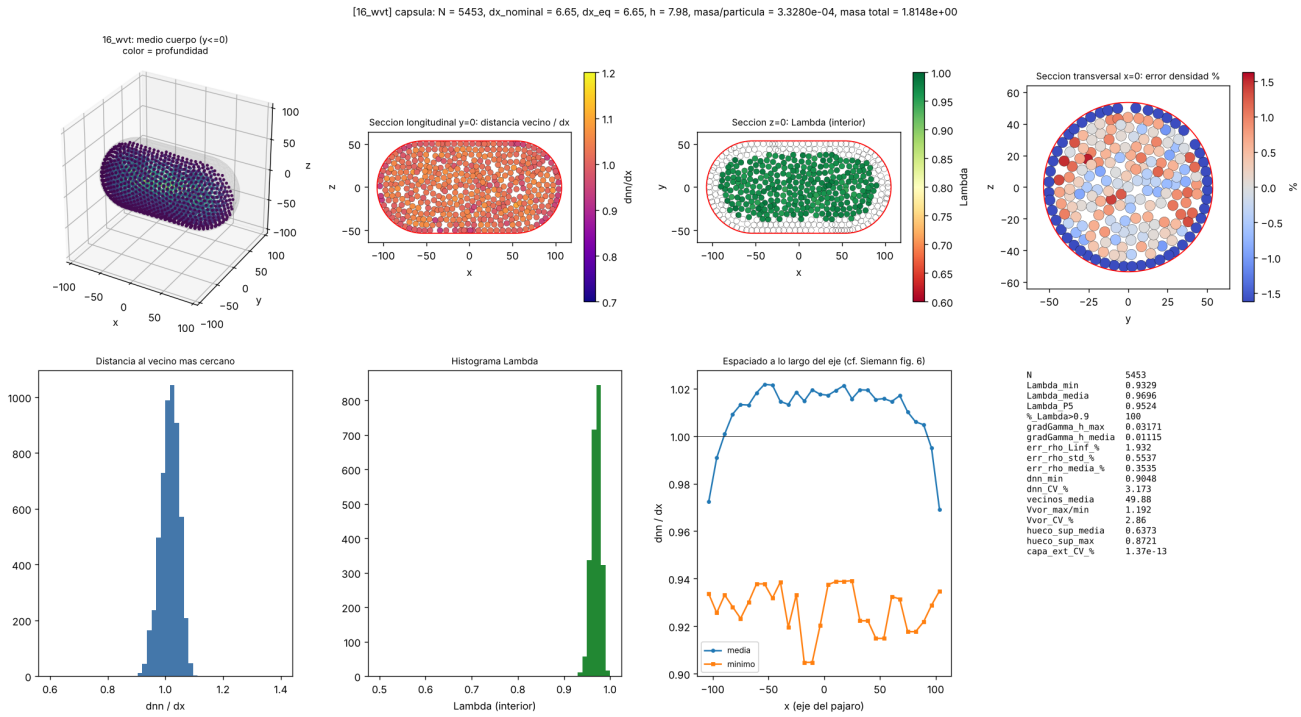


Figura 5. Resumen 3D del método WVT sobre el pájaro.

7.2 Ventana interactiva (VISOR_INTERACTIVO = True)

- Deslizador **corte**: mueve el plano de corte. A la izquierda se ven solo las partículas a un lado del plano (el interior del cuerpo cortado); a la derecha, la sección a tamaño real.
- Deslizador **espesor/dx**: grosor de la loncha de la sección.
- Botones **campo** (Λ, dnn/dx, error de densidad, |∇Γ|h, profundidad) y **eje de corte** (x, y, z).
- La vista 3D se gira arrastrando con el ratón.

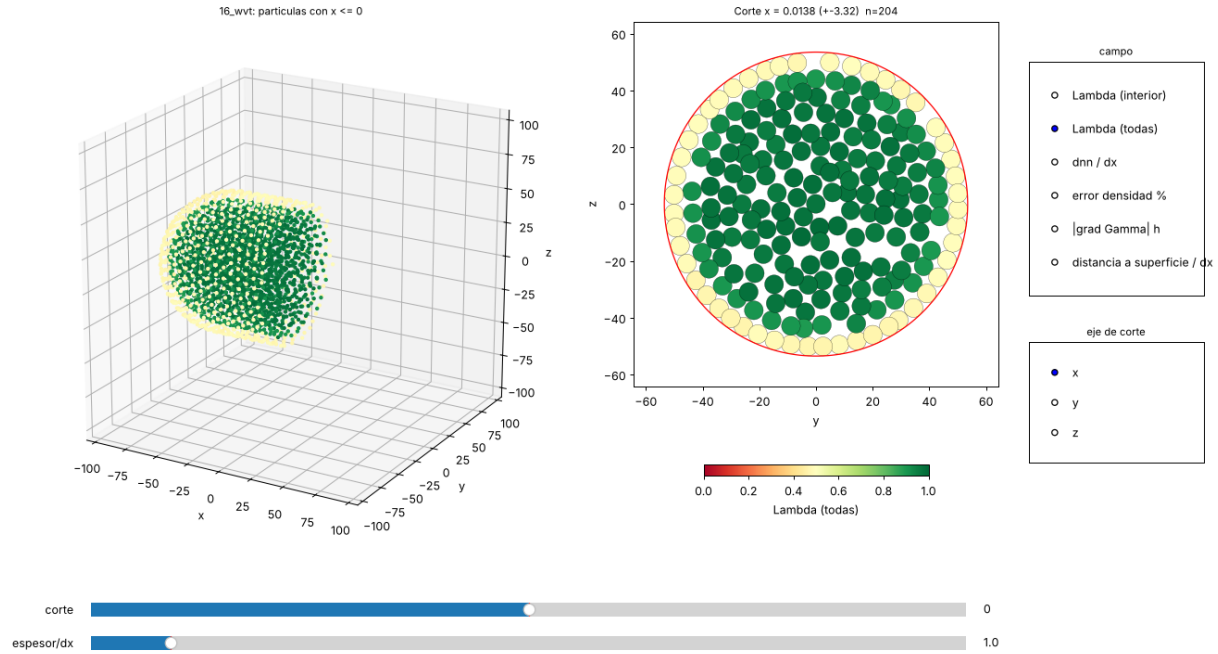


Figura 6. Ventana interactiva de matplotlib.

7.3 Visor HTML (fichero _visor3D.html en resultados)

Se abre con doble clic en Chrome, Edge o Firefox. No necesita internet ni instalar nada. Es el más fluido para revisar el volumen y dibuja las partículas como esferas de diámetro dx.

Acción	Cómo
Girar	Botón izquierdo del ratón
Desplazar	Botón derecho, o Mayús + izquierdo
Zoom	Rueda
Reiniciar vista	Doble clic
Corte	Eje (X/Y/Z) + modo: semiespacio ≤ / ≥ (ver el interior) o loncha (sección de espesor elegido)
Campo de color	Λ (interior o todas), dnn/dx, error de densidad, |∇Γ|h, profundidad
Otros	Tamaño de partícula, superficie en alambre, métricas desplegables

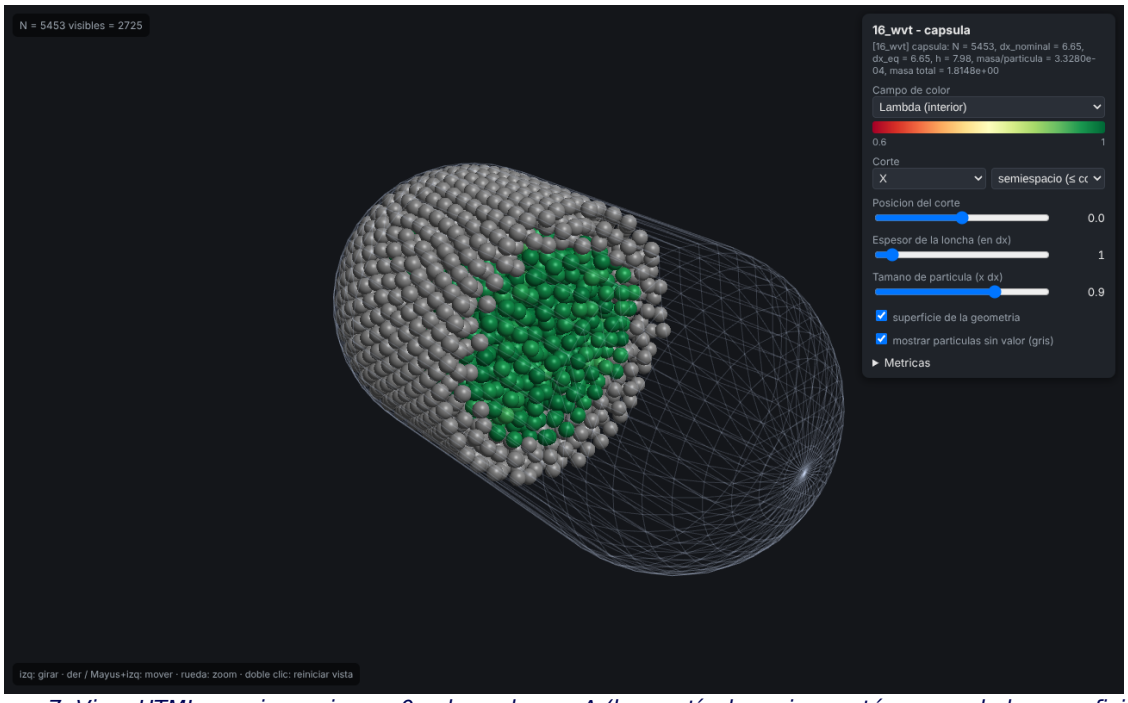


Figura 7. Visor HTML: semiespacio x ≤ 0 coloreado con Λ (las partículas grises están cerca de la superficie).

También se genera un **.vtk** para ParaView con masa, h, Λ, densidad, ∇Γ, dnn, vecinos y distancia a la superficie como campos.

8. Usar un fichero .stl

- 1. Copia el fichero en la carpeta **stl** del proyecto.
- 2. Ejecuta **19_comprobar_stl.py** con ARCHIVO = "stl/tu_fichero.stl". Tiene que decir **malla cerrada SI** y un volumen y un tamaño razonables en mm.
- 3. En cualquier script 3D (11–18):

```
GEOMETRIA = "stl"
PARAMS_GEOM = {"archivo": "stl/tu_fichero.stl", "escala": 1.0, "centrar": True}
```

- Formatos: STL ASCII y binario. La malla debe ser **cerrada** (estanca). Si tiene agujeros, el script 19 avisa; repárala en HyperMesh o ANSA.
- **escala** multiplica las coordenadas (1000 si el STL está en metros). **centrar** mueve el centro de masas al origen; con False las partículas quedan en las coordenadas originales del CAD.
- La dirección de impacto es la que traiga el STL: oriéntalo antes de exportarlo.

- Cómo funciona: distancia exacta a la malla (punto más cercano sobre el triángulo más cercano) y signo dentro/fuera con pseudonormales de cara, arista y vértice, que es robusto en zonas cóncavas. Validado contra la fórmula exacta: error ≤ 0,1 mm, que es lo que se separa la malla de triángulos de la superficie real.
- Ejemplos incluidos: `stl\ejemplo_pajaro_capsula.stl` y `stl\ejemplo_toro.stl` (cóncavo).
- Con STL, los métodos de empaquetado tardan algo más (unos 5–10 min con 5.500 partículas).

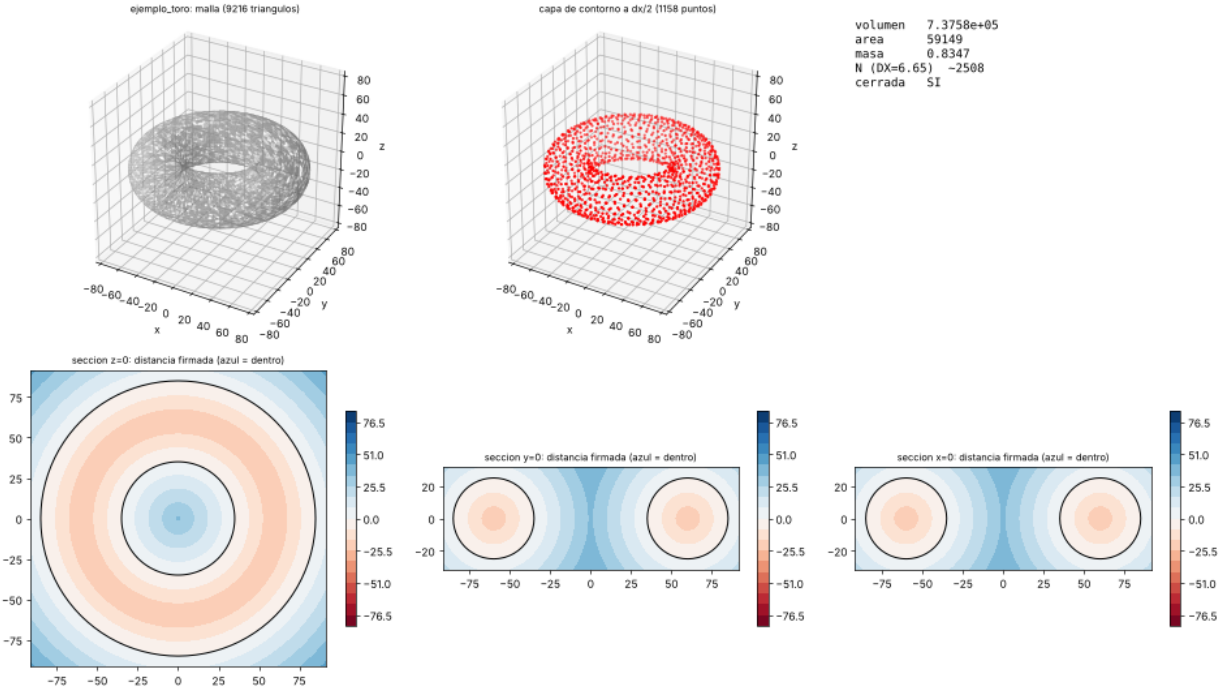


Figura 8. Script 19 con el toro de ejemplo: malla, capa de contorno y secciones de la distancia firmada.

9. Exportación a PAM-CRASH

Cada script escribe `resultados\<geom>_<metodo>_pamcrash.inc` en formato de columnas fijas:

```
NODE  /       1      -104.14947      -9.3224891             0.
...
SPHEL /       1       1       16.077E-5
        |IDEL  |IDPRT |IDNOD |MASS
```

Tarjeta	Columnas	Campos
`NODE /`	9-16 | 17-32 | 33-48 | 49-64	IDNOD | X | Y | Z
`SPHEL /`	9-16 | 17-24 | 25-32 | 33-40	IDEL | IDPRT | IDNOD | MASA (igual para todas)

- La cabecera del fichero (líneas $) incluye N, dx_eq, la h sugerida, ρ0, la masa por partícula y la total.
- Al final hay una plantilla **PART tipo SPHEL comentada** para completar con tu material (EOS Tait/Murnaghan), RATIO/Hmin/Hmax del smoothing length, etc.
- La masa va en 8 caracteres (p. ej. `6.077E-5`), con unos 4 dígitos significativos.
- **Comprueba los campos con el manual de tu versión de PAM-CRASH/VPS antes de lanzar**: el formato está sacado de la sintaxis estándar, sin el manual oficial delante.
- En 2D se exporta con z = 0 (sirve para inspeccionar, no para simular).

10. La librería sph_common (por dentro)

Los scripts son cortos porque todo el trabajo está en esta carpeta. No hace falta tocarla para usar el proyecto, pero conviene saber dónde está cada cosa:

Módulo	Contenido principal
`geometria2d.py`	Geometria2D (polígono denso), distancia firmada exacta, curvas paralelas (offset), esquinas; crear_geometria()
`geometria3d.py`	Geometria3D basada en la función de distancia firmada (SDF); esfera, caja, cilindro, cápsula, elipsoide; muestreo uniforme de superficies paralelas; crear_geometria_3d()
`geometria_stl.py`	Lector STL ASCII/binario, distancia exacta a la malla, signo con pseudonormales, volumen, comprobación de malla cerrada
`redes2d.py / redes3d.py`	Redes cúbica, FCC, hexagonal/panal (2D); SC, BCC, FCC, HCP (3D), recortadas a la geometría
`kernels.py`	Wendland C2 (soporte 2h) y quintic spline (soporte 3h), en 2D y 3D
`vecinos.py`	Búsqueda de pares con cKDTree; listas de Verlet con «piel» para los bucles iterativos
`packing.py`	empaquetar_colagrossi, empaquetar_negi, empaquetar_wvt (válidos en 2D y 3D)
`metricas.py`	calcular_metricas(): Λ, Γ, ∇Γ, densidad, espaciado, Voronoi, superficie
`particulas.py`	Clase DistribucionSPH: posiciones, masa igual, h, dx_eq, guardar/cargar .npz
`visor.py / visor3d.py`	Figuras 2D; resumen 3D con secciones, ventana interactiva y panel de comparación 3D
`visor_html.py`	Visor WebGL autocontenido (.html)
`exportar.py`	PAM-CRASH (.inc), CSV y VTK
`flujo.py`	procesar(): métricas, figuras, guardado y exportación; lo llaman todos los scripts

Esquema de cualquier script de método:

```
geom = crear_geometria_3d(GEOMETRIA, **PARAMS_GEOM)        # 1. geometria
x, hist, info = empaquetar_wvt(geom, DX, ...)              # 2. posiciones
dist = DistribucionSPH(x, geom, "16_wvt", DX, rho0=RHO0)  # 3. masa, h
procesar(dist, hist=hist)                       # 4. metricas, figuras, export.
```

11. Problemas frecuentes

Síntoma	Causa probable / solución
ModuleNotFoundError: sph_common	No estás ejecutando desde la carpeta del proyecto. Abre la carpeta en VS Code o haz cd a ella
ModuleNotFoundError: numpy / scipy	pip install -r requirements.txt (en el mismo Python que usa VS Code)
El script parece «colgado» al final	Están abiertas las ventanas de matplotlib: ciérralas (o MOSTRAR = False)
El comparador recalcula todo	Has cambiado DX o la geometría respecto a lo guardado, o falta el .npz
Tarda mucho	Sube DX (N ∝ 1/dx³) o baja MAX_ITER para pruebas; con STL es más lento
El visor HTML sale en blanco	El navegador tiene WebGL desactivado; prueba con Chrome o Edge
AVISO STL: aristas abiertas	La malla no es estanca: repárala antes (HyperMesh, ANSA, Meshmixer)
Tamaño del STL raro	Unidades: usa escala = 1000 si está en metros
Sección vacía o rara en una red	La loncha se centra en la fila de partículas más cercana; cambia el espesor en el visor

12. Referencias

- [1] J.J. Monaghan, *Smoothed particle hydrodynamics*, Rep. Prog. Phys. 68 (2005) 1703.
- [2] R. Vacondio et al., *Grand challenges for Smoothed Particle Hydrodynamics numerical schemes*, Comp. Part. Mech. 8 (2021) 575–588.
- [3] M.S. Shadloo, G. Oger, D. Le Touzé, *Smoothed particle hydrodynamics method for fluid flows, towards industrial applications*, Computers & Fluids 136 (2016) 11–34.
- [4] M.H. Siemann, S.A. Ritt, *Novel particle distributions for SPH bird-strike simulations*, CMAME 343 (2019) 746–766.
- [5] A. Colagrossi, B. Bouscasse, M. Antuono, S. Marrone, *Particle packing algorithm for SPH schemes*, CPC 183 (2012) 1641–1653.
- [6] P. Negi, P. Ramachandran, *Algorithms for uniform particle initialization in domains with complex boundaries*, CPC (2021), arXiv:1910.07898; código: gitlab.com/pypr/sph_geom.
- [7] S. Diehl et al., *Generating optimal initial conditions for SPH simulations*, PASA 32 (2015).